

FOR JAVA DEVELOPERS WHO CAN NEARLY READ A NESTJS FILE

The Twenty Percent That Bites

Spring Boot to TypeScript, episode 1

Your instincts are right about eighty percent of this. The rest looks identical.

This is a warm up, not a language course. Every item earns its place by answering one question: does it help you read, change or ship a NestJS service?

The beat worth keeping is the container. An interface does not exist at runtime, so NestJS cannot inject by type, so every provider you write needs a token. That single chain explains most of what looks like ceremony in a NestJS codebase.

The versions on this sheet were current on 22 September 2026: TypeScript 6.0.3 in the NestJS project, 7.0.2 in the plain Node project, Node 22.22.2 and NestJS 12.0.3. The decorator behaviour in particular has changed across TypeScript versions, so check the date on anything else you read about it.

In scope

- Classes, and the constructor parameter shorthand
- private readonly against private final, and what it does not guarantee
- Why the container refuses an interface, and what a token fixes
- Structural typing against nominal typing
- Promises against threads, and the one thread that serves everybody
- Decorators as functions that have already run
- Imports by file path, and the .js suffix that looks wrong

Out of scope

- Generics beyond reading them
- Conditional types, mapped types and the utility types
- Namespaces, declaration merging and the compiler API
- Validation itself, which has its own episode



- The declared field type that validates nothing

A warm up, so the framework is readable. If you want the language properly, take a language course, and you will get more out of it after this.

01 What transfers, what does not, and what is measured

Constructor injection	transfers completely
private final	becomes a compile time note only
Injecting by interface	does not transfer. Use a token
The interface, in the compiled file	0 occurrences
What the container reads instead	design:paramtypes = [Object]
Nominal typing	does not transfer. Shape decides
A declared field type	validates nothing at all
Versions measured on	TypeScript 6.0.3, Node 22.22.2, NestJS 12.0.3

Every figure on the right came off a terminal on 22 September 2026. Reproduce all of it with scripts/verify-di-tokens.sh.



03 private readonly is not private final

The constructor shorthand declares the field and assigns it in one place, which is the thing you miss when you go back to Java.

But readonly is checked by the compiler and then erased. Nothing at runtime defends the field, so a cast is enough to write to it.

payment.ts

```
export class Payment {  
  constructor(public readonly id: string) {}  
}
```

```
const p = new Payment('p_1');  
(p as any).id = 'MUTATED';  
console.log(p.id); // MUTATED
```

Compiles clean. The compiler refuses a direct write and cannot stop the cast.

readonly is a compile time check with no runtime representation.

TypeScript handbook, Type Compatibility and Modules, checked 2026-09-22.

04 Why the container will not build

This is the most ordinary code in Spring. It compiles with no errors and the application does not start.

The reason is one line of the compiled output. A decorator makes the compiler record what the constructor takes, so the container can read it back. After erasure there is no interface left to record, so it records `Object`, and `Object` is not something the container has a provider for.

by-type.ts, which does not start

```
@Injectable()
class PaymentsService {
    constructor(private readonly gateway: PaymentGateway) {}
}

// Nest can't resolve dependencies of the PaymentsService (?).
// Make sure the argument at index [0] is available in the module.
```

what the compiler emitted for it

```
PaymentsService = __decorate([
    (0, common_1.Injectable)(),
    __metadata("design:paramtypes", [Object])
], PaymentsService);
```

The interface name appears zero times in the compiled file.

with-token.ts, which does. Copy this wiring.

```
const PAYMENT_GATEWAY = Symbol('PaymentGateway');

@Injectable()
class PaymentsService {
  constructor(
    @Inject(PAYMENT_GATEWAY)
    private readonly gateway: PaymentGateway,
  ) {}
}

@Module({
  providers: [
    {provide: PAYMENT_GATEWAY, useClass: StripeGateway},
    PaymentsService,
  ],
})
class AppModule {}

// charged 10000
```

Nothing recovered the interface. The constructor asks for a value that exists at runtime instead of a type that does not.

Nest refuses with: can't resolve dependencies of the PaymentsService (?). Please make sure that the argument at index [0] is available in the current module.

Run on a real machine, 2026-09-22, by scripts/verify-di-tokens.sh in the course repository.

A provider token may be a string or a symbol, and a symbol avoids collisions between modules.

NestJS documentation, Custom Providers, checked 2026-09-22.

05 Shape decides compatibility, not the name

Java compares the name of the type. Two types are compatible when one declares that it is the other. TypeScript asks only whether the shape fits.

This is the design rather than a hole in it, and it matters the first time you pass the wrong identifier into the right shaped parameter.

shapes.ts

```
class Merchant { constructor(public id: string) {} }  
class Customer { constructor(public id: string) {} }
```

```
const m: Merchant = new Customer('c_1');  
console.log(m.id); // c_1
```

The same assignment is a compile error in Java.

TypeScript uses structural typing: compatibility is decided by members, not by declared inheritance.

TypeScript handbook, Type Compatibility and Modules, checked 2026-09-22.

06 An annotation is data. A decorator already ran.

A Java annotation is metadata, inert until something reads it reflectively. A decorator is a function, and it runs when the file loads, before the class has finished being defined and long before any instance exists.

At the time of writing NestJS uses the older decorator behaviour, switched on by `experimentalDecorators`, with `emitDecoratorMetadata` supplying the type metadata the container reads.

nestjs-api/tsconfig.json, the two options that matter

```
{  
  "compilerOptions": {  
    "emitDecoratorMetadata": true,  
    "experimentalDecorators": true,  
    "module": "nodenext",  
    "target": "ES2023"  
  }  
}
```

`emitDecoratorMetadata` is what writes `design:paramtypes`. Without it the container has nothing to read at all.

Decorators are evaluated when the class definition is evaluated, not when an instance is created.

TypeScript handbook, Type Compatibility and Modules, checked 2026-09-22.

07 The declared type validates nothing

In Spring, Jackson coerces at the boundary or returns a 400, so the declared type is enforced before your method runs. In NestJS nothing binds: the JSON becomes an object and that object reaches you as it arrived.

What makes this dangerous rather than annoying is that it is broken by operator. Addition concatenates; multiplication coerces and works.

the same field, two adjacent lines

```
typeof amount : string
amount + 1    : 100001
amount * 2    : 20000
```

Every test that multiplies passes. The one place you add a fee quietly grows a digit.

A declared field type has no runtime representation and performs no validation.

TypeScript handbook, Type Compatibility and Modules, checked 2026-09-22.

Validation at the boundary is separate code, configured as a pipe.

NestJS documentation, Custom Providers, checked 2026-09-22.

Before you write a NestJS provider

1. If the dependency is an interface, give it a token and inject the token
2. Prefer a symbol over a string token, so two modules cannot collide
3. Do not trust a declared field type to have validated anything
4. Assume readonly is a compile time note, not a runtime guarantee
5. Write the .js suffix on relative imports, and expect it to look wrong
6. Never block the event loop: there is one thread serving every request



Everything else

Site

<https://code-with-sam-dev.github.io>

YouTube

<https://www.youtube.com/@CodewithSam-Dev>

LinkedIn

<https://www.linkedin.com/company/code-with-sam-dev>

X

<https://x.com/CodeWithSamDev>

TikTok

<https://www.tiktok.com/@codewithsamdev>

Instagram

<https://www.instagram.com/codewithsamdev>

GitHub

<https://github.com/code-with-sam-dev>

If this was useful, a like and a subscribe help more than you would think. And tell me in the comments which part of your workflow you want taken apart next.

Node.js is a trademark of the OpenJS Foundation. Java is a trademark of Oracle. Spring and Spring Boot are trademarks of Broadcom. NestJS and TypeScript are the property of their respective owners. This is an independent, unofficial guide produced by Code with Sam, not affiliated with or endorsed by any of them, and no third party artwork is reproduced here.

Technical claims verified 2026-09-22 against the documentation named beside each one. Vendors change behaviour and defaults between versions, so check the version you are actually running before you rely on any of it.